

Other hardening

The catalogue and pcode locks, the logs, line endings, and the rest of the smaller changes.

Everything on this page is a change you may notice while testing, grouped by what it touches. The identity model and the file permissions are on Security; this is the remainder.

The global catalogue

Adding to or removing from the system-wide catalogue now requires administrator rights, whichever way you ask for it.

`catalog` already required them for the spelled-out form, `catalog bp myprog global`. **It did not require them for the form most people use** — putting a `*`, `!`, `_` or `$` in front of the name. `delete.catalog` required nothing at all, by either route.

This matters because the system-wide catalogue holds the programs SD runs for everybody, `$login` among them. **Replacing one ran your code in every session on the machine, administrators included; deleting one stopped everybody signing in.**

Nothing changes for local and private cataloguing, which is what programmers use day to day:

```
catalog bp myprog          private catalogue, this account
catalog bp myprog local    this account's VOC
```

Both still work, in any account you are allowed to `logto` into. The only thing an ordinary user can no longer do is catalogue a program whose name starts with `*`, `!`, `_` or `$` — those characters mean *system-wide*. Name it without one.

To catalogue system-wide you need an elevated session, not just an administrator account. Start SD from an elevated window and `logto sdsys`. Entering SDSYS from an ordinary window still gives you administrator rights *inside* SD, but **Windows itself now refuses the write** — so use the same elevated window you already need in order to recompile the system programs.

The pcode library

`<sysdir>\bin` holds the pcode library — the interpreter itself, which SD loads into shared memory at start-up and every session then runs.

UNTIL 23 Aug 2026 ANY MEMBER OF `sdusers` COULD WRITE TO IT, so one SD user could have replaced what everybody else's session executes, including an administrator's.

It is now readable by SD users and writable only by administrators. Nothing needs to write it after an install — only the process that starts SD reads it, and that is already elevated. **If you have anything that writes into `<sysdir>\bin`, it will now be refused and will need to run elevated.**

Scheduled jobs

A scheduled task can run an SD command without administrator rights, and only the commands an administrator has named for it. The permit list is the SDSYS file `batch.jobs`, locked read-only to SD users by the same control as the `os.users` list.

It has its own page: **Scheduled jobs**.

The logs

There are three, and they are not interchangeable.

FILE	WHERE	FOR
<code>audit</code>	<code>C:\ProgramData\SD\sdsys</code>	who did what — logins, refusals, <code>logto</code> , grants. See Security and the operating system
<code>errlog</code>	<code>C:\ProgramData\SD\sdsys</code>	diagnostics, and API connection records
<code>sd-elevate.log</code>	<code>C:\ProgramData\SD</code>	what the elevation helper actually did

`SD-ELEVATE.LOG`

It records when a helper started for a session, each script it was asked to run, the exit code that came back, and when it stopped.

Only administrators can read or write it. Ordinary SD users are not on its permissions at all. That is different from the audit trail, which SD users *can* add to because SD writes it as them; nothing unelevated ever writes this one.

It is a diagnostic, not the audit trail. For *who obtained privilege and when*, read `audit`. This file answers "*the account was not created — what actually happened*", which previously had no answer at all.

If the file is missing, nothing is logged and SD does not create one. That is deliberate: a log created on the fly would inherit permissions letting every SD user rewrite it, and **a record of privileged work that its own subjects can edit is worse than none**. A reinstall keeps whatever is already there.

THE ERROR LOG RECORDS WHO CONNECTS TO THE API PORT

Every accepted API connection adds a line naming the Windows process and account at the other end:

```
API connection from 127.0.0.1:59314 - pid 11448, GITORLI\don
```

Nothing is refused on the strength of it. This records who connected; it does not decide who may. The API's own checks are unchanged.

A connection forwarded over ssh shows `sshd`, not the person at the far end. The tunnel ends on this machine, so the process that connects genuinely is `sshd`. What the line distinguishes is a client running *on* this machine from one arriving through a tunnel; **it cannot name a remote person.**

This matters beyond the log. SD refuses an administrator an API session from another computer, and it decides that from the address the connection came from. A tunnelled connection arrives from `127.0.0.1` because it really does start here, so **it is admitted**. If that matters to you, turn port forwarding off in `sshd_config`; no check inside SD can see through a tunnel.

"peer process not identified" means the client had already gone by the time the connection was looked up. It is not an error and the connection proceeds normally.

TWO THINGS ABOUT ERROR-LOG TRIMMING

ERRLOG now applies to these lines too. The background daemon used to append without ever trimming — it only wrote at start-up and on failure, so it never grew. Now that it writes per connection, it discards the oldest part of the log on reaching the `ERRLOG` size in `sd.conf`. **If you have set `ERRLOG` unusually large, consider what an entry per connection adds to it.**

After the log is trimmed, its first line may have no timestamp. An entry is two lines — a timestamped header and the message indented below it — and trimming restarts the file at a **line**, not at an entry, so the first message can be left without its header. **This is not damage** and no entry after it is affected. SD has always trimmed this way; it is only visible now because the log turns over more often.

Line endings

Both halves are fixed, and they were fixed separately.

READING — FILES EDITED IN NOTEPAD OR SAVED FROM EXCEL

Directory files exist so you can edit their records with an ordinary Windows editor, and Windows editors end each line with CR+LF. **SD only ever looked for the LF, so it kept the CR — and put it on the end of the data.**

You would have seen it as an invisible extra character at the end of every line: comparisons failing for no visible reason, a name that would not match, a trailing space that was not a space.

Reading a CSV saved by Excel is the clearest case. The last column of every row picked up the stray character, because a comma ended the other columns and the line ending only ever touched the last one. `READCSV`, `READSEQ` and reading a directory file record are all corrected.

A CR on its own is still data and is left exactly as it is. Only the CR+LF pair that ends a line is treated as a line ending, so this cannot alter data that happens to contain a CR.

WRITING

Anything SD writes that an ordinary Windows program can open now ends its lines with CR+LF: records in a directory file, `WRITESEQ` and `WRITECSV` output, command output captured with `COMO`, printer output sent to a file, and the error log.

SD's CSV statements are documented as following RFC 4180, and **that standard asks for CR+LF**.

Dynamic files are unaffected — they are stored in SD's own format and are not readable by other programs.

Existing files are left alone, so a file can contain both endings. SD reads either, so this is untidy rather than a problem.

The terminal

The default terminal type is now `WINDOWS`. `TERM` on its own should say `Device : windows`.

THE ARROW KEYS DID NOTHING in cmd, PowerShell or Windows Terminal on earlier builds. A terminal has two spellings for an arrow key: in its ordinary state it sends `ESC [ D` for Left, and the other spelling `ESC O D` only after the application asks it to switch — **which SD never does**. The `vt100` definition SD was defaulting to lists only the second spelling, so SD was listening for a key no Windows console ever sends.

The shipped `WINDOWS` definition is an exact copy of `LINUX`, which had this right all along — its name describes an operating system, but what matters is the byte protocol.

Existing accounts keep their old setting until their VOC is updated. An upgrade now does that for every account, and `update.accounts` does it on demand. Until then, `term windows` sets it for the session. **63 definitions ship, compiling to 100 terminal names** — the extra names are variants such as `vt100-w` and `vt220-at` — so `term wyse60` still works.

A name that is not installed is refused and your current type is kept — *"Unrecognised terminal name"* — so a typo costs you nothing. `term` **with no argument reports the type actually in force**, which is how to check.

Watch for near-misses all the same. There is no plain `vt320` — the shipped name is `vt320-at`. `terminfo.src` ships with SD, so `sdtic` can add a definition that is not there.

Backspace works, at the prompt and when you are asked for a password.

THE PAGE IS 120 × 36, NOT 80 × 24

SD's default terminal size is 120 columns by 36 lines. It is not a cosmetic default: the shipped `@` dictionary records and the default `list` report layouts are formatted for 120 columns. **A console window narrower than that makes ordinary reports look wrapped or truncated**, which reads as a formatting bug and is not one.

`term` reports the size in force, above the `Device` line:

```
:term
Page width: 120
Page depth: 36
Device      : windows
```

The size is worked out at login, in this order: the `LINES` and `COLUMNS` environment variables if they are numeric, otherwise the terminfo entry's `lines` and `cols`, **otherwise 36 and 120** — then raised to a minimum of 10 × 20 if smaller. So a console or ssh session normally gets its real window size and 120 × 36 is the fallback when nothing answers, which is the case for a phantom or a piped script.

`term default` **restores it, and it prints nothing when it does.** It sets the same 120 × 36 the login path falls back to and returns silently, so run a bare `term` after it to see the result. `term 120,36` does the same by hand.

If you have notes from an earlier build, this is one of the things that changed: `term default` used to set 20 × 24 — the *minimum* width and a fixed depth rather than the defaults — so it made the display worse instead of putting it back.

Paths

A Windows PATH typed at the command prompt is no longer cut off at the first backslash. Typing `C:\Data\Sales` was read as just `C:`, with the rest treated as a second, separate thing. `create.account other`, which is given a folder to put the account in, was the command most likely to show it — the account went to the wrong place, or the command failed for a reason that made no sense from what you had typed.

Forward slashes always worked and still do. **Both are now read the same way.**

PowerShell execution policy — leave it alone

This is a hardening page, so it is worth saying plainly: **tightening your PowerShell execution policy does not break SD, and loosening it does not help SD.** Set it to whatever your own security policy wants.

SD does a lot of its Windows-side work — `logto sdsys`, `append.sd.path`, `remote.api`, `remote.ssh` and the editor verbs — by running a small PowerShell script that the installer put in `C:\Program Files\SD`. Each one is launched with `-ExecutionPolicy Bypass` **on that single command line**, which applies to that one process and that one script. It does

not change the machine, and it does not affect any other script you or anyone else runs.

If you loosened the policy to get SD working, put it back. That was needed on builds before 5 September 2026, where those commands failed with *"running scripts is disabled on this system"*. It is fixed. The `sh` verb still gives you a PowerShell prompt under your machine's own policy — SD lifts the restriction only for the scripts it installed itself, never for a shell you type into.

The one case that does stop SD is Group Policy. A policy that sets the execution policy outranks anything a program can pass on a command line. Run `Get-ExecutionPolicy -List`: if `MachinePolicy` or `UserPolicy` reads `Restricted` or `AllSigned`, SD's administrative commands will fail and only your Windows administrator can change it. `Undefined`, `RemoteSigned`, `Unrestricted` or `Bypass` on those two rows are all fine. **The Installed Scripts page in the Administrator set has the detail.**

Running SD

The service	String Database (SD)
After an unclean shutdown	SD now starts anyway, rather than refusing because the last stop was abrupt
Nested sessions	SD will not start a second time inside itself
<code>sd <command></code>	needs an elevated session, or an entry in <code>batch.jobs</code> — see above

Setting no password

Earlier builds of this port had both the end of the installer and SD's own prompt say that without a password you could not use ssh or the API, and stop there. **That was true and easy to read as "some things will not work".**

It is stronger than that, and both now say so: with no password the account can be used **only at that computer** — at the keyboard, or through Remote Desktop or similar remote-control software — **and only from a session run as administrator.**

You can still choose it, and SD asks again the next time you open the account.